

Slovenská technická univerzita v Bratislave

Fakulta elektrotechniky a informatiky

Technická dokumentácia

IoT webová aplikácia na monitorovanie teploty a vlhkosti a
riadenie ventilátora

Predmet: POIT

Autor: Ablazov Yehor

Akademický rok: 2025/2026

Bratislava

Obsah

1	Úvod	2
2	Cieľ zadania	2
3	Použitý hardvér a softvér	3
3.1	Použitý hardvér	3
3.2	Použitý softvér	3
4	Návrh architektúry systému	3
5	UML diagramy	4
5.1	UML diagram prípadov použitia	4
5.2	UML komponentový diagram	5
5.3	UML sekvenčný diagram	5
6	Hardvérové zapojenie	6
6.1	Zapojenie snímača DHT11	7
6.2	Zapojenie relé modulu a ventilátora	7
7	Firmvér ESP32	8
8	Komunikačný protokol	8
9	Serverová časť	9
9.1	Databáza SQLite	9
9.2	Archivácia do CSV súboru	10
10	Klientska časť	10
10.1	Grafy a ručičkové ukazovatele	10
11	Používateľská príručka	10
11.1	Spustenie systému	10
11.2	Práca s aplikáciou	10
12	Vývojárska príručka	11
12.1	Inštalácia závislostí	11
12.2	Nahratie firmvéru	11
13	Verziónovanie a odovzdanie	11
14	Záver	11

1. Úvod

Cieľom záverečného zadania je vytvoriť webovú aplikáciu, ktorá umožňuje monitorovanie alebo riadenie signálov získaných z reálnych senzorov. Vytvorený systém má spĺňať koncepciu IoT, teda získavať údaje z fyzického zariadenia, prenášať ich cez sieť, ukladať ich a zobrazovať používateľovi prostredníctvom webového rozhrania.

V rámci tejto práce bol vytvorený systém na meranie teploty a relatívnej vlhkosti vzduchu pomocou snímača DHT11 pripojeného k mikrokontroléru ESP32. Namerané hodnoty sú odosielané cez Wi-Fi sieť na serverovú aplikáciu vytvorenú v jazyku Python pomocou frameworku Flask. Serverová aplikácia zabezpečuje príjem údajov, ich archiváciu v databáze SQLite, zápis do súboru CSV a poskytovanie údajov klientskému webovému rozhraniu.

Súčasťou systému je aj jednoduché riadenie ventilátora pomocou relé modulu. Ventilátor sa zapína a vypína podľa nastavenej teplotnej hranice. Regulácia je realizovaná dvojpolohovo s hystereziou, aby sa predišlo častému prepínaniu relé pri malých zmenách teploty.

2. Cieľ zadania

Cieľom projektu je navrhnúť a realizovať webovú aplikáciu, ktorá bude umožňovať:

- inicializáciu systému tlačidlom *Open*,
- nastavenie parametrov monitorovania a regulácie,
- spustenie monitorovania tlačidlom *Start*,
- zobrazenie nameraných údajov vo forme zoznamu,
- zobrazenie údajov vo forme grafov,
- zobrazenie údajov pomocou ručičkových ukazovateľov,
- archiváciu údajov do databázy,
- archiváciu údajov do súboru,
- zastavenie monitorovania tlačidlom *Stop*,
- ukončenie systému tlačidlom *Close*.

Projekt využíva reálny hardvér, konkrétne vývojovú dosku ESP32, snímač DHT11, relé modul a 5V ventilátor. Z tohto dôvodu nejde iba o simulačné riešenie, ale o praktickú IoT aplikáciu s reálnym meraním a jednoduchým riadením akčného člena.

3. Použitý hardvér a softvér

3.1. Použitý hardvér

Komponent	Účel v projekte
ESP32 NodeMCU 38-pin s CP2102	Mikrokontrolér s Wi-Fi pripojením, zber údajov zo snímača a odosielanie údajov na server.
DHT11	Digitálny snímač teploty a relatívnej vlhkosti vzduchu.
Relé modul 5V	Spínanie ventilátora podľa signálu z ESP32.
Ventilátor 5V	Akčný člen použitý na jednoduché chladenie pri prekročení teplotnej hranice.
Nepájivé pole	Realizácia zapojenia bez spájkovania.
Prepojovacie vodiče	Prepojenie ESP32, DHT11, relé modulu a ventilátora.
USB-C kábel	Napájanie ESP32 a nahrávanie firmvéru z Arduino IDE.

Tabuľka 1: Použitý hardvér

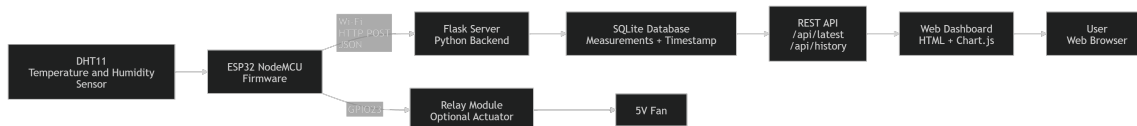
3.2. Použitý softvér

Softvér / technológia	Účel
Arduino IDE	Vývoj a nahratie firmvéru do ESP32.
Python	Implementácia serverovej časti.
Flask	Webový server a REST API.
SQLite	Lokálna databáza pre archiváciu meraní.
CSV súbor	Archivácia meraní do súboru.
HTML, CSS, JavaScript	Klientska časť webovej aplikácie.
Chart.js	Vykreslenie historických grafov.
GitHub	Verzovanie projektu a odovzdanie zdrojových súborov.

Tabuľka 2: Použitý softvér

4. Návrh architektúry systému

Systém je rozdelený na tri hlavné časti: hardvérovú, serverovú a klientskú. Hardvérovú časť tvorí ESP32 so snímačom DHT11, relé modulom a ventilátorom. Serverová časť je vytvorená vo frameworku Flask a zabezpečuje komunikáciu s ESP32, ukladanie dát do databázy a poskytovanie dát pre webové rozhranie. Klientska časť je webový dashboard, ktorý umožňuje ovládanie systému a vizualizáciu nameraných údajov.



Obr. 1: Architektúra systému

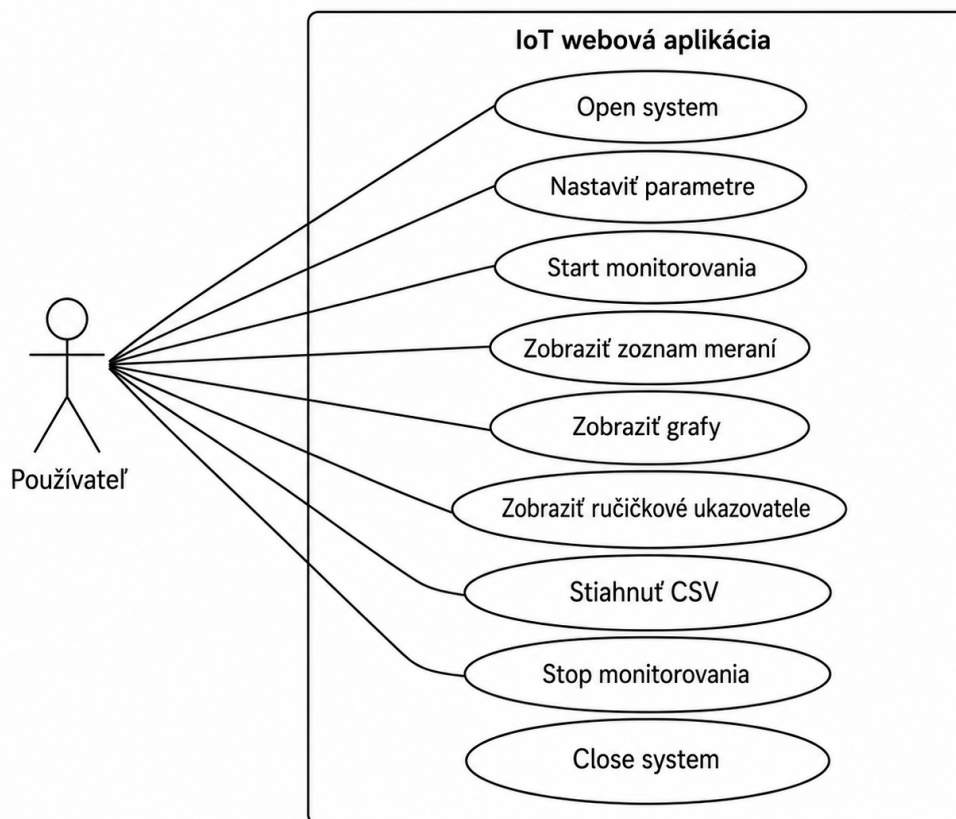
Tok údajov v systéme je nasledovný:

1. Snímač DHT11 meria teplotu a vlhkosť.
2. ESP32 číta hodnoty zo snímača a riadi ventilátor podľa nastavenej logiky.
3. ESP32 odosiela údaje na Flask server pomocou HTTP POST požiadavky vo formáte JSON.
4. Flask server údaje spracuje a uloží ich do SQLite databázy a CSV súboru.
5. Webový dashboard získava aktuálne a historické údaje zo servera cez REST API.
6. Používateľ sleduje hodnoty, grafy a ručičkové ukazovatele v prehliadači.

5. UML diagramy

5.1. UML diagram prípadov použitia

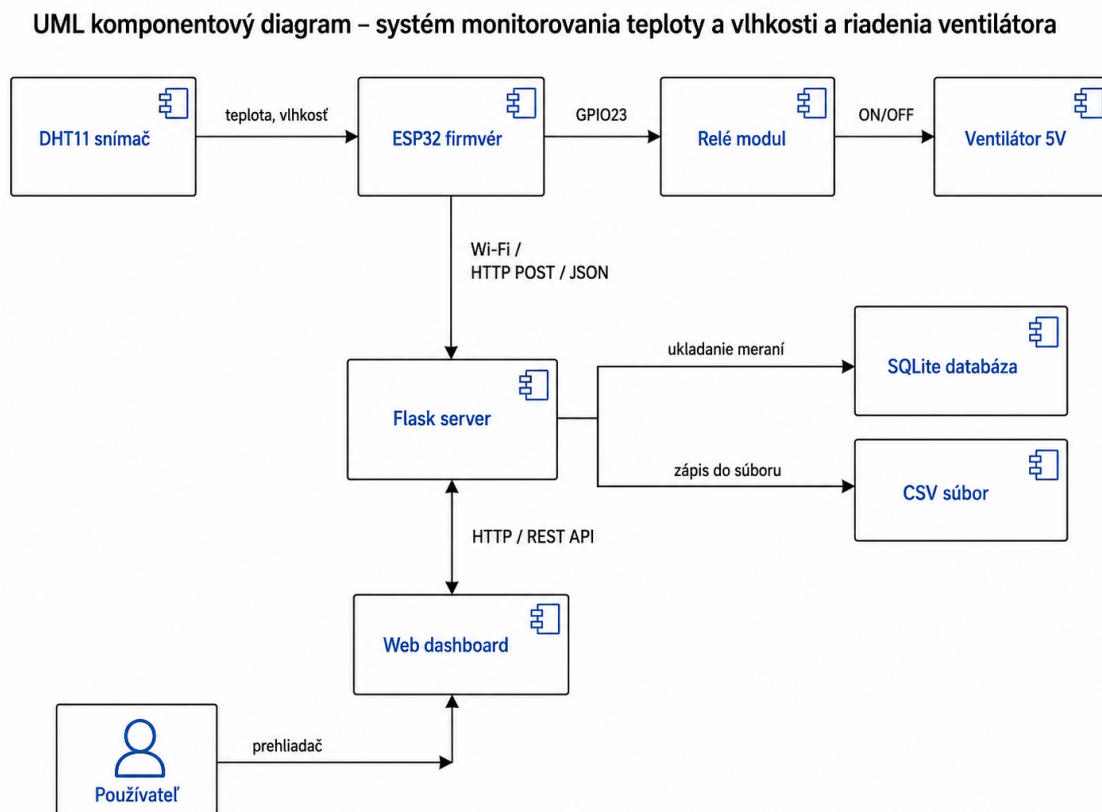
Diagram prípadov použitia zobrazuje hlavné funkcie systému z pohľadu používateľa. Používateľ môže inicializovať systém, nastaviť parametre, spustiť a zastaviť monitorovanie, sledovať údaje vo forme zoznamu, grafov a ručičkových ukazovateľov, stiahnuť CSV súbor a ukončiť systém.



Obr. 2: UML diagram prípadov použitia

5.2. UML komponentový diagram

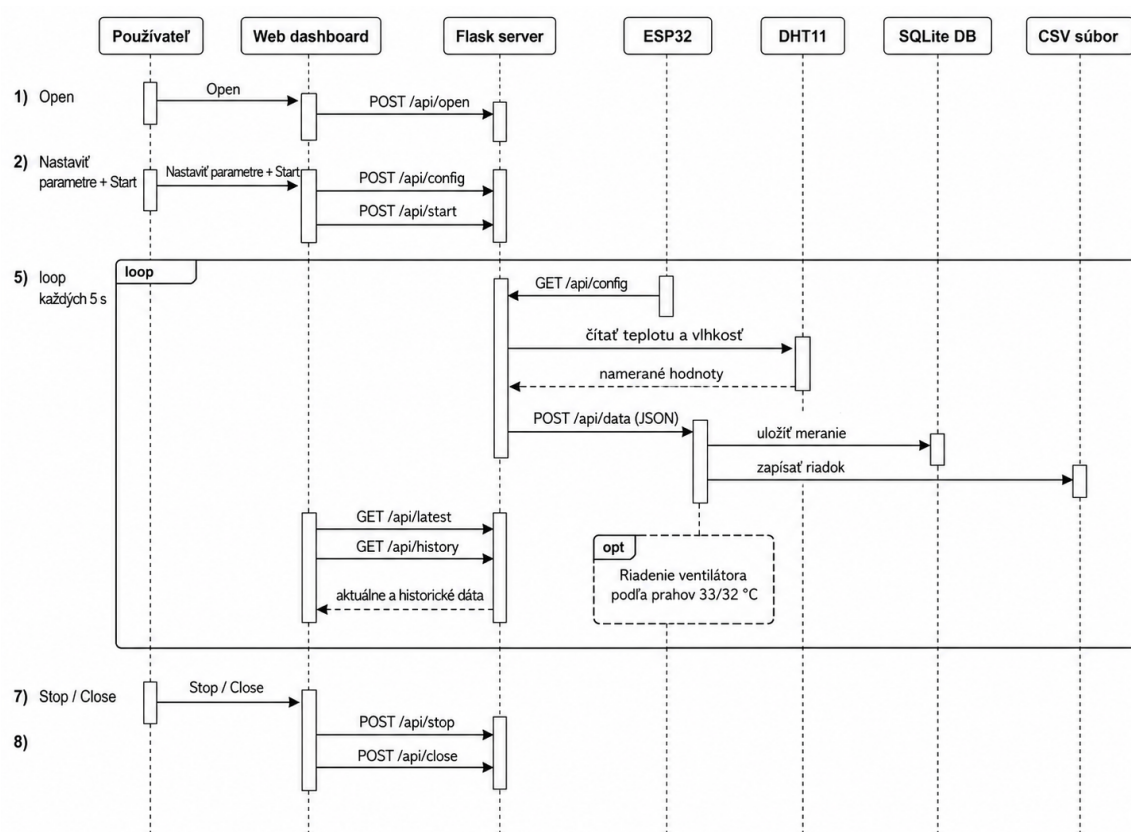
Komponentový diagram zobrazuje hlavné časti systému a ich vzájomné prepojenie. ESP32 komunikuje so snímačom DHT11 a relé modulom. Serverová aplikácia Flask prijíma údaje cez Wi-Fi sieť, ukladá ich do databázy SQLite a do CSV súboru. Webový dashboard komunikuje so serverom pomocou REST API.



Obr. 3: UML komponentový diagram

5.3. UML sekvenčný diagram

Sekvenčný diagram opisuje typický priebeh komunikácie v systéme. Používateľ najskôr otvorí systém, nastaví parametre a spustí monitorovanie. ESP32 potom v pravidelných intervaloch číta údaje zo snímača, získava konfiguráciu zo servera a odosiela namerané hodnoty. Server údaje ukladá do databázy a súboru, pričom dashboard si ich pravidelne načítava a zobrazuje.

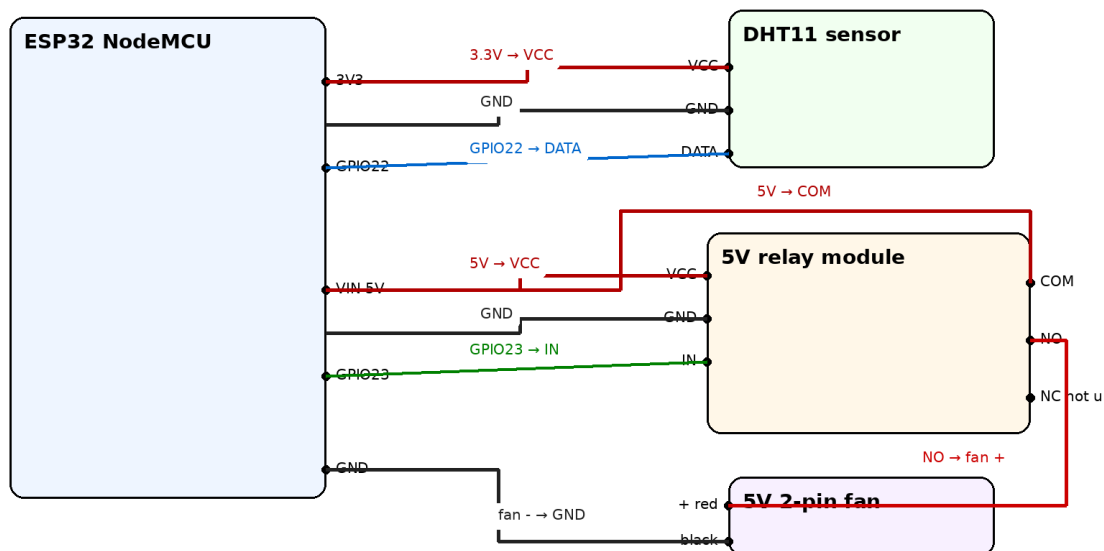


Obr. 4: UML sekvenčný diagram komunikácie

6. Hardvérové zapojenie

Snímač DHT11 je pripojený k ESP32 pomocou troch vodičov: napájanie, zem a dátový vodič. Dátový vodič je pripojený na GPIO22. Relé modul je pripojený na GPIO23 a slúži na spínanie ventilátora. Ventilátor je pripojený cez kontakty COM a NO relé modulu, aby bol v pokojovom stave vypnutý.

Wiring schema: ESP32 + DHT11 + Relay + 5V Fan



Obr. 5: Schéma zapojenia hardvéru

6.1. Zapojenie snímača DHT11

DHT11	ESP32
VCC	3V3
GND	GND
DATA	GPIO22

Tabuľka 3: Zapojenie snímača DHT11

6.2. Zapojenie relé modulu a ventilátora

Relé modul / ventilátor	Zapojenie
VCC relé	VIN 5V na ESP32
GND relé	GND na ESP32
IN relé	GPIO23
COM relé	5V
NO relé	kladný vodič ventilátora
záporný vodič ventilátora	GND

Tabuľka 4: Zapojenie relé modulu a ventilátora

Relé modul je aktívny v logike LOW. To znamená, že relé zopne pri privedení logickej nuly na vstup IN. Vo firmvéri sa preto používa režim podobný open-drain: pre zapnutie ventilátora sa pin nastaví ako výstup a privedie sa LOW, pre vypnutie ventilátora sa pin nastaví ako vstup.

7. Firmvér ESP32

Firmvér je vytvorený v prostredí Arduino IDE. Jeho úlohou je inicializovať snímač DHT11, pripojiť ESP32 k Wi-Fi sieti, čítať namerané hodnoty, riadiť ventilátor a odosielať údaje na server.

Hlavné funkcie firmvéru:

- pripojenie k Wi-Fi sieti,
- čítanie teploty a vlhkosti zo snímača DHT11,
- pravidelné získavanie konfigurácie zo servera,
- dvojpolohová regulácia ventilátora s hystereziou,
- tvorba JSON správy,
- odosielanie údajov na server pomocou HTTP POST.

Regulácia ventilátora je realizovaná podľa týchto prahov:

- pri teplote vyššej alebo rovnjej 33 °C sa ventilátor zapne,
- pri teplote nižšej alebo rovnjej 32 °C sa ventilátor vypne.

Použitá hysterezia zabráňuje častému prepínaniu relé pri teplote blízkej rozhodovacej hranici.

8. Komunikačný protokol

Komunikácia medzi ESP32, serverom a webovým dashboardom je realizovaná pomocou HTTP protokolu. ESP32 odosiela namerané hodnoty na server pomocou požiadavky POST na endpoint:

```
1 POST /api/data
```

Dáta sú prenášané vo formáte JSON. Príklad správy odosielanej z ESP32:

```
1 {  
2   "temperature": 30.20,  
3   "humidity": 44.00,  
4   "unit_temperature": "C",  
5   "unit_humidity": "%",  
6   "sensor": "DHT11",  
7   "fan_state": "OFF"  
8 }
```

Webový dashboard komunikuje so serverom pomocou REST API endpointov:

Endpoint	Metóda	Popis
/	GET	Zobrazenie webového dashboardu.
/api/open	POST	Inicializácia systému.
/api/start	POST	Spustenie monitorovania.
/api/stop	POST	Zastavenie monitorovania.
/api/close	POST	Ukončenie systému.
/api/config	GET/POST	Získanie alebo nastavenie parametrov regulácie.
/api/data	POST	Príjem údajov z ESP32.
/api/latest	GET	Získanie posledného merania.
/api/history	GET	Získanie historických meraní.
/api/file-history	GET	Získanie údajov archivovaných v CSV súbore.
/api/download-csv	GET	Stiahnutie CSV súboru.

Tabuľka 5: REST API endpointy

9. Serverová časť

Serverová časť je implementovaná v jazyku Python pomocou frameworku Flask. Server prijíma údaje z ESP32, spracúva používateľské požiadavky z webového rozhrania, ukladá údaje do databázy a zapisuje ich do súboru.

Server obsahuje stav systému, ktorý určuje, či je systém otvorený a či je monitorovanie aktívne. Dáta z ESP32 sa ukladajú iba vtedy, keď používateľ najskôr stlačí tlačidlo *Open* a následne *Start*. Tlačidlo *Stop* zastaví ukladanie nových meraní a tlačidlo *Close* deaktivuje systém.

9.1. Databáza SQLite

Na archiváciu údajov je použitá databáza SQLite. Výhodou SQLite je, že nevyžaduje samostatný databázový server a údaje sú uložené v jednom lokálnom súbore.

Tabuľka `measurements` obsahuje tieto stĺpce:

Stĺpec	Typ	Popis
id	INTEGER	Primárny kľúč.
temperature	REAL	Nameraná teplota.
humidity	REAL	Nameraná relatívna vlhkosť.
sensor	TEXT	Názov použitého snímača.
unit_temperature	TEXT	Jednotka teploty.
unit_humidity	TEXT	Jednotka vlhkosti.
fan_state	TEXT	Stav ventilátora.
fan_on_threshold	REAL	Prah zapnutia ventilátora.
fan_off_threshold	REAL	Prah vypnutia ventilátora.
timestamp	TEXT	Čas prijatia merania.

Tabuľka 6: Štruktúra tabuľky measurements

9.2. Archivácia do CSV súboru

Okrem databázy sa údaje zapisujú aj do CSV súboru `measurements.csv`. Tento súbor obsahuje čas merania, teplotu, vlhkosť, názov snímača, jednotky, stav ventilátora a nastavené prahy regulácie. Používateľ si môže CSV súbor stiahnuť priamo z webového rozhrania.

10. Klientska časť

Klientská časť je realizovaná ako webový dashboard v jazykoch HTML, CSS a JavaScript. Webová stránka komunikuje so serverom pomocou REST API a pravidelne obnovuje aktuálne hodnoty.

Dashboard obsahuje:

- tlačidlá Open, Start, Stop a Close,
- formulár na nastavenie prahov regulácie a intervalu merania,
- aktuálnu teplotu a vlhkosť,
- stav ventilátora,
- čas posledného merania,
- graf teploty,
- graf vlhkosti,
- ručičkové ukazovatele,
- tabuľku posledných meraní,
- možnosť zobrazit' údaje zo súboru CSV,
- možnosť stiahnuť CSV súbor.

10.1. Grafy a ručičkové ukazovatele

Historické dáta sú zobrazované pomocou grafov. Používateľ si môže vybrať medzi režimom posledných 20 meraní a režimom za posledných 24 hodín. Okrem grafov sú aktuálne hodnoty zobrazené aj pomocou ručičkových ukazovateľov, ktoré poskytujú rýchlu vizuálnu informáciu o stave meraných veličín.

11. Používateľská príručka

11.1. Spustenie systému

Používateľ spustí serverovú aplikáciu príkazom:

```
1 cd server
2 python3 app.py
```

Následne otvorí webový prehliadač a zadá adresu:

```
1 http://127.0.0.1:5000/
```

Ak sa používa vzdialený prístup alebo tunelovanie, použije sa adresa poskytnutá príslušnou službou.

11.2. Práca s aplikáciou

Používateľ vykonáva prácu so systémom v tomto poradí:

1. Stlačí tlačidlo *Open*, čím inicializuje systém.
2. Nastaví parametre regulácie: teplotu zapnutia ventilátora, teplotu vypnutia ventilátora a interval merania.
3. Stlačí tlačidlo *Start*, čím spustí monitorovanie.
4. Sleduje aktuálne hodnoty, grafy, ručičkové ukazovatele a tabuľku meraní.
5. Podľa potreby stiahne CSV súbor.
6. Tlačidlom *Stop* zastaví monitorovanie.
7. Tlačidlom *Close* ukončí prácu so systémom.

12. Vývojárska príručka

12.1. Inštalácia závislostí

Na inštaláciu serverovej časti je potrebné vytvoriť virtuálne prostredie a nainštalovať závislosti:

```
1 cd server
2 python3 -m venv venv
3 source venv/bin/activate
4 pip install -r requirements.txt
```

Súbor `requirements.txt` obsahuje minimálne:

```
1 Flask
```

12.2. Nahratie firmvéru

Firmvér sa nahráva do ESP32 pomocou Arduino IDE. Pred nahratím je potrebné nastaviť Wi-Fi údaje a adresu servera:

```
1 const char* WIFI_SSID = "YOUR_WIFI_SSID";
2 const char* WIFI_PASSWORD = "YOUR_WIFI_PASSWORD";
3 const char* SERVER_BASE_URL = "http://YOUR_SERVER_IP:5000";
```

Z bezpečnostných dôvodov sa reálne heslá a lokálne IP adresy nemajú ukladať do verejného repozitára.

13. Verzionovanie a odovzdanie

Pri vývoji bol použitý verzionovací systém GitHub. Repozitár obsahuje zdrojové súbory firmvéru, serverovej časti, klientskej časti, dokumentáciu, schémy a UML diagramy.

Adresa repozitára:

```
1 https://github.com/orgen229/Misa
```

14. Záver

V rámci projektu bola vytvorená IoT webová aplikácia na monitorovanie teploty a vlhkosti a jednoduché riadenie ventilátora. Systém využíva reálny hardvér, bezdrôtovú komunikáciu, serverovú časť, databázovú a súborovú archiváciu a webové rozhranie s viacerými spôsobmi vizualizácie.

Projekt spĺňa požiadavky zadania, pretože obsahuje inicializáciu systému, nastavenie parametrov, spustenie a zastavenie monitorovania, zoznam meraní, grafy, ručičkové ukazovatele, archiváciu do databázy aj súboru a ukončenie systému. Použitý návrh je rozšíriteľný napríklad o presnejší snímač, autentifikáciu, HTTPS komunikáciu alebo nasadenie servera do cloudu.